

Schichten-Architektur, Transaktionen 2016-06-08

Grundlegende Entwurfsentscheidungen

- Programmiersprache: {Scala, Java, C++, C#, VB, PHP, Skriptsprachen, ...}
- UI-System: {Java: Swing|JavaFX, Web: JSF|Vaadin, ...}
- DB-System: {JPA | JDBC | MongoDB | db4o ...}

Entwurfsprinzipien für Komponenten

- Minimale Schnittstellen
- Austauschbarkeit $\Rightarrow$ Zusammenhängendes in einer Komponente, Schnittstelle als Interface
- Testbarkeit $\Rightarrow$ möglichst als API (Application Programming Interface: Unterprogrammschnittstelle)
- Wiederverwendbarkeit

Ergebnis: Schichten-Architektur

Paket	Inhalt
l1_ui	User Interface: Reine Darstellung, Ereignisbehandlung, I/O; ohne Verarbeitung
↓	
l2_lg	Geschäftslogik (Fachkonzept) als API: Prüfungen, Folgeoperationen, Buchungen;
↓	ohne UI, DB-Aspekte, ohne SQL
l3_da	Data Access: Datenbankzugriffe als API: Reines Speichern/Suchen; ohne
↓	Verarbeitung
l4_dm	Data Model: Dumme Datenbehälter

Regeln

- l1-ui-Schicht exportiert nur prüfende Operationen, garantiert die Einhaltung der Geschäftsregeln.
- l1-ui-Schicht darf nicht auf l3-da-Schicht zugreifen, damit die Prüfungen nicht umgangen werden können.
- l1-ui-Schicht sollte die Prüfungen durch l2-lg-Schicht nicht duplizieren (Bsp. Plausi-Prüfungen).

Vom Analysemodell zum Entwurf

Aus einer in der Analyse gefundenen Klasse (*Bsp.* Konto) werden im Entwurf Klassen in allen Schichten.

- In der **Oberflächenschicht** ist typischerweise je Dialog eine Klasse nötig (*Bsp.* UiKonto, UiKontoÜberweisung). Außerdem Klassen für wiederkehrende Elemente von Dialogen (*Bsp.* UiKontoList, UiButton).
- In die **Logikschicht** wird die Fachklasse mit ihren Operationen übernommen (*Bsp.* LgKonto), um die Objektverwaltungsoperationen sowie bei Bedarf ergänzt oder in Interface und Implementierung aufgespalten. [Jede Klassenoperation (*Bsp.* erfassen, suchen) sollte dabei durch Verlagerung in ihre Ausgangsklasse zu einer Objektoperation umgewandelt werden.]
- In der **Datenzugriffsschicht** haben wir [für Sprachen mit Reflection wie Java] eine generische Klasse DaGeneric (Generic DAO), die für eine beliebige Dm-Klasse deren Attribute persistent machen kann. Außerdem benötigen wir gemäß [Siedersleben] ein Interface, das den operativen Zugriff auf den Persistenzmanager kanalisiert. Wir nehmen dafür die DaFactory.
[In Sprachen ohne Reflection wie C++ kann man DaBase nicht generisch lösen, sondern muß je Fachklasse eine Da-Klasse schreiben/generieren, die die Attribute der OOA-Klasse und die Operationen daInsert(), daUpdate() und daDelete() enthält.]
- In die **Datenmodellschicht** wird die Fachklasse aus der Analyse mit allen ihren Attributen übernommen (*Bsp.* DmKonto). Diese Klassen enthalten meist keine Logik. Sie dienen als "Value Objects" bzw. "Data Transfer Objects" zwischen Logikschicht und Datenbank, aber auch zwischen Logikschicht und Oberflächenschicht.

Transaktion in der Datenzugriffsschicht, Logikschicht oder Oberflächenschicht?

Bsp.: Die Operation

```
quellKonto.überweisen(io_zielKonto, i_betragCt);
```

besteht geschäftsregelmäßig aus den Schritten

```
quellKonto.saldoCt -= i_betragCt;
```

```
io_zielKonto.saldoCt += i_betragCt;
```

```
daKonto.save(quellKonto);
```

```
daKonto.save(zielKonto);
```

Ziel: Durchführung **ganz** oder **gar nicht**!

Transaktionen gehorchen dem **ACID-Prinzip**: Atomic, Consistent, Isolated, Durable (Datenbank-Stoff).

Zur Unterstützung von Transaktionen beim Datenbankzugriff gibt es in **JDBC** in der Klasse `java.sql.Connection` folgende Operationen:

- `commit()`: Abspeichern der bisherigen Änderungen sichtbar für Andere
- `rollback()`: Verwerfen der Änderungen seit dem letzten `commit()`, im Fehlerfall aufzurufen.

Wo Transaktion definieren?

Gemäß [Siedersleben] spezifizieren wir die Logikschicht in einer heilen Welt, d.h. als ob es nur einen Systembenutzer gäbe. Die Probleme des wechselseitigen Ausschlusses paralleler Benutzer und des Ganz-oder-Garnicht-Durchführens von Veränderungen können mit dem Transaktionskonzept gelöst werden.

In welcher Schicht entscheiden, welche Methodenaufrufe zu einer Transaktion zusammenfassen?

[Antwort: Nicht DA. UI ist gut, aber nicht mit JPA, damit Prüfungen nicht umgangen werden können $\Rightarrow$ Lg]

Bsp. Heile Welt in der Klasse LgKonto:

```
public void überweisen(  
    final LgKonto zielKonto, final int betragCt  
) {  
    //Veränderungen im Arbeitsspeicher vornehmen (eigentliche Geschäftslogik):  
    this        .saldoCt -= betragCt;  
    zielKonto.saldoCt += betragCt;  
  
    //Veränderungen gegen Fremdänderungen prüfen;  
    //in DB isoliert (nur für sich selbst) sichern:  
    daKonto.save(this);  
    daKonto.save(io_zielKonto);  
} //überweisen
```

Transaktionsabsicherung:

Jede externe Operationen muss als Transaktion formuliert sein; dies macht man, indem man bei Erfolg am Ende `commit()`, andernfalls `rollback()` aufruft. Es ist m.E. sinnvoll, diese Operationsaufrufe in die selbst geschriebenen Operationen `lgSession.beginTransaction()` und `lgSession.endTransaction()` zu kapseln, um auch noch andere transaktionsbegleitende Aktionen automatisch mit auszuführen.

Bsp.

```
final int betrag = ...//von Oberfläche
_lgSession.beginTransaction();
boolean ok = false;
try {
    _quellKonto.überweisen(_zielKonto, betrag);
    ok = true; //Erfolg vermerken
} finally{ //Veränderungen freigeben für andere oder verwerfen je nach ok
    _lgSession.endTransaction(ok);
}
```

Bem.: Der **finally**-Zweig wird bei jeder Beendigung des **try**-Blocks durchgeführt, sei es erfolgreich oder fehlerhaft.. Als Transaktionsabschluss muß `lgApplication.endTransaction` je nach dem `ok` –Wert `commit()` oder `rollback()` aufrufen.

Achtung: Bei JPA müssen Entitäten nach Transaktionsende DETACHED werden, bevor sie an die Oberfläche gegeben werden!

Achtung: Zwecks Redundanzvermeidung muss die Transaktionssteuerung nach dem Template Method Pattern oder mittels Dynamic Proxy zentralisiert werden.